



UNIVERSIDADE DA CORUÑA

First lab assignment: programming agents with JADE

**Daniel Jove Díaz
Adriano Miranda Seoane**

Master in Artificial Intelligence
Multi Agent Systems

A Coruña, March 2025

Contents

Contents	i
1 Summary	1
2 Implementation	2
2.1 Main Classes Used	2
2.2 Methods	2
2.2.1 General Behavior of the Participant Agents	2
2.2.2 Communication Behaviors	4
2.2.3 Pre-Conditions, State Descriptions, and Specific Parameters	5
2.2.4 Reactive vs. Deliberative Approaches	5
3 Simulation experiments	6
3.1 A fixed world without traps	6
3.2 A fixed world with 10 traps	6
3.3 A dynamic world without traps	6
3.4 A dynamic world with 10 traps	7
4 Discussion of the results and conclusions	8
4.1 Performance in Fixed Worlds	8
4.2 Performance in Dynamic Worlds	8
4.3 Pros and Cons of Each Agent	8
4.4 Effect of Commitment Parameter	9
4.5 Best Overall Agent	9

1. Summary

This project presents the development of a multi-agent system in which three autonomous agents compete in a discrete grid environment to collect items and avoid traps. The simulation is managed by a simulator agent responsible for coordinating turns, receiving the agents' actions, and updating the state of the environment.

Each of the three agents has a unique behavior, influencing its strategy for exploration, item collection, and obstacle avoidance. Throughout the simulation, agents must communicate with the simulator to report their decisions and receive information about the environment. The ultimate goal is to maximize the accumulated score by the end of the game, determining the winner.

This project explores key concepts of multi-agent systems, such as autonomous decision-making, agent communication, and interaction in competitive environments. Different behavioral approaches and their impact on agent efficiency within the system are analyzed.

2. Implementation

For the implementation of this multi-agent system, we used JADE (Java Agent Development Framework), a framework that facilitates the development of agent-based applications in Java, leveraging object-oriented programming principles.

2.1 Main Classes Used

- **Map**: Represents the grid environment where agents navigate, placing items, traps, and defining boundaries.
- **Participant**: Defines the structure of an agent, including its properties and communication methods.
- **SimulatorAgent**: Manages the simulation by handling turns, processing agent actions, and updating the game state.
- **SimulationManagerBehaviour**: Controls the overall execution flow of the simulation.
- **RegisterParticipantsBehaviour**: Handles the registration process of participating agents.
- **RegisterRequestBehaviour**: Manages agents' requests to register within the system.
- **RegisterActionBehaviour**: Processes and validates actions submitted by agents.
- **RandomBehaviour**: A behavior followed by one of the agents, moving randomly through the map without any strategy.
- **GreedySearchBehaviour**: A behavior where an agent selects the shortest path to the nearest item at each round but lacks long-term planning.
- **BlindSnakeBehaviour**: A predefined movement pattern where the agent follows a sequence of moves, changing direction when encountering a visited position or an obstacle.
- **GetInitialStateBehaviour**: Handles the reception of the initial simulation state. It updates the agent's state and assigns it a predefined behavior, such as **GreedySearchBehaviour**.

JADE provides built-in functionalities for agent communication, behavior management, and scheduling, allowing for a modular and scalable design of our competitive multi-agent system.

2.2 Methods

2.2.1 General Behavior of the Participant Agents

The participant agents in this simulation use a variety of behaviors to interact with the environment. These behaviors drive their decision-making processes and allow them to communicate with each other and the simulator. Below, we describe the specific decision functions of each agent type and how they interact with the simulation environment.

BlindSnakeBehaviour

This agent attempts to move in a fixed directional pattern, choosing from four cardinal directions (right, up, left, and down). It continues changing direction until it finds an unvisited position. The agent's decision is based on avoiding previously visited positions and resetting the visited positions if it cannot find a new valid move after several attempts.

- **Decision Function:** The agent follows a reactive strategy, moving in a fixed direction until it encounters an obstacle. If the agent cannot find an unvisited position after multiple attempts, it resets the visited positions and retries.
- **Adequacy:** This reactive approach makes the agent simple and effective in environments with few obstacles. However, it may struggle in more complex or dynamic environments where more sophisticated path planning is required.

GreedySearchBehaviour

This agent employs a goal-seeking behavior, calculating the Euclidean distance to the nearest resource and selecting the move that minimizes this distance. It avoids traps and attempts to find the nearest target.

- **Decision Function:** The agent computes the distance to each neighboring position and selects the move that brings it closest to the nearest resource. If no resources are found, the agent resorts to a random move. Moreover, a strategy to avoid repeated positions has been implemented in order to avoid getting stuck in some special situations that we have experimented throughout the development of this assignment.
- **Adequacy:** The deliberative nature of this behavior allows the agent to make informed decisions. However, the agent is focused on short-term goals and may fail to consider the long-term dynamics of the environment, such as traps or other agents' movements.

RandomBehaviour

This agent chooses its movements randomly from a list of possible directions (up, down, left, right). It does not evaluate the environment or consider its goals when making decisions, making it a purely reactive agent.

- **Decision Function:** The agent selects a random direction to move without evaluating the environment.
- **Adequacy:** This approach can be seen as inefficient for more complex tasks but works in simple or exploratory environments. Its behavior is entirely reactive, without any strategic planning.

RegisterParticipantsBehaviour

This behavior is responsible for managing the participation of agents in the simulation. When an agent requests to join, this behavior ensures that there is space for it and provides the initial simulation state to the joining agent.

- **Decision Function:** When an agent requests to join, the `RegisterParticipantsBehaviour` checks if there is space for more participants. If so, it adds the agent to the simulation and provides an initial state. If not, it refuses the request.
- **Adequacy:** This is a basic management behavior ensuring that the system can handle multiple agents joining or leaving the simulation in a controlled manner.

RegisterRequestBehaviour

This behavior allows a `ParticipantAgent` to send a request to the `SimulatorAgent` to join the simulation. It includes the agent's commitment level as part of the request message.

- **Decision Function:** The agent sends a message to the `SimulatorAgent` with its commitment level, asking to join the simulation. The response from the `SimulatorAgent` determines whether the agent is added to the simulation.
- **Adequacy:** This behavior allows for proper agent registration and is necessary to establish the agents within the simulation, ensuring proper synchronization between agents and the simulator.

RequestActionBehaviour

This behavior involves the agent receiving requests for its next move and sending back its proposed action. It either selects a random move or a move based on its decision-making strategy.

- **Decision Function:** The agent receives a request for its next action, selects a move from its predefined list of directions (up, down, left, right), and sends the proposed move back as a response.
- **Adequacy:** This behavior handles communication between agents and the simulator, making it essential for the functioning of the simulation. It is reactive, as it simply responds to incoming requests without strategic planning.

2.2.2 Communication Behaviors

Communication between agents and the simulator is central to the operation of the multi-agent system. The following behaviors manage the interactions using the JADE communication framework:

Cyclic Behaviour

All agent behaviors are implemented using `CyclicBehaviour`, which ensures that the agents continuously listen for incoming messages, process them, and take action accordingly. This is ideal for handling repetitive communication patterns such as requesting and sending actions or states.

- **Message Handling:** Agents use `MessageTemplate` to filter messages based on conversation ID and performative. For example, in `RequestActionBehaviour`, the agent listens for `REQUEST` messages and sends back a `PROPOSE` with its chosen move.

Message Types

- **REQUEST:** Agents send requests to ask for their next action or to request updates.
- **PROPOSE:** The agent proposes its next move or action in response to a request.
- **INFORM:** After performing an action, the agent sends an update to inform others of its new state.
- **AGREE / REFUSE:** Used in registration behaviors to confirm or deny participation requests.

2.2.3 Pre-Conditions, State Descriptions, and Specific Parameters

Each agent has specific parameters and states that influence its behavior:

- **Position:** Each agent has a position on the grid, which is central to its movement decisions.
- **SimulationState:** The agent's state, including its current position, map, and other simulation variables, is stored in `SimulationState`. The state is updated after every action.
- **Map:** The map defines the grid layout, including obstacles, traps, and resource positions. The agents evaluate the map to decide on their next action.
- **Commitment:** Each agent has a level of commitment, which might influence its behavior (e.g., `RegisterRequestBehaviour` involves sending the agent's commitment to the `SimulatorAgent`).

2.2.4 Reactive vs. Deliberative Approaches

- **BlindSnakeBehaviour:** Reactive. The agent does not plan its actions in advance and only reacts to its immediate environment by changing direction based on its surroundings.
- **GreedySearchBehaviour:** Deliberative. This behavior involves a more thoughtful decision-making process, as the agent calculates the best move toward the nearest resource.
- **RandomBehaviour:** Reactive. It responds randomly to requests, making it very simplistic with no consideration of the environment or objectives.

3. Simulation experiments

3.1 A fixed world without traps

Type of agent	Average Final Score			
	Agent alone (commitment 1)	Agent alone (commitment 20)	All 3 together (commitment 1)	All 3 together (commitment 20)
Random Agent	18.8	20.2	23	25.6
Greedy Agent	198	180	196.6	149.2
Snake Agent	67.3	68	59.6	62.2

3.2 A fixed world with 10 traps

Type of agent	Average Final Score			
	Agent alone (commitment 1)	Agent alone (commitment 20)	All 3 together (commitment 1)	All 3 together (commitment 20)
Greedy Agent	190.4	176	144	198.3
Random Agent	-83.3	-89.1	-59.6	-54.2
Snake Agent	-105.2	-81	-159.3	-68.6

3.3 A dynamic world without traps

Type of agent	Average Final Score			
	Agent alone (commitment 1)	Agent alone (commitment 20)	All 3 together (commitment 1)	All 3 together (commitment 20)
Greedy Agent	162.6	53.6	160.6	54.3
Random Agent	35.0	35.1	38.3	36.6
Snake Agent	53.3	46.3	51.3	49.3

3.4 A dynamic world with 10 traps

Type of agent	Average Final Score			
	Agent alone (commitment 1)	Agent alone (commitment 20)	All 3 together (commitment 1)	All 3 together (commitment 20)
Greedy Agent	138	-399.3	140.5	-343.75
Random Agent	-84.6	-68.3	-76	-58.75
Snake Agent	-39.2	-60.3	-83.3	-71

4. Discussion of the results and conclusions

The results obtained from the different scenarios provide valuable insights into the performance of the three agent behaviors: GreedySearchBehaviour, RandomBehaviour, and BlindSnakeBehaviour in a game where three participants must navigate a grid, collecting items that increase their score while avoiding traps that decrease it.

4.1 Performance in Fixed Worlds

In the world without traps, the Greedy Agent demonstrated superior performance, as expected, achieving the highest average scores. By prioritizing items efficiently, it was able to maximize its score, while the Random and Snake Agents displayed lower but stable performance. The Random Agent moved unpredictably, leading to occasional advantages but also inefficiencies, while the Snake Agent, with its structured movement pattern, scored somewhere in between the other two methods, demonstrating that structured movement across the entire map allows it to find objects better than random movement despite the lack of awareness of where objects are located.

When traps were introduced into the fixed world, unawareness strategies (Random and Blind Snake) had a huge negative impact on their performance since these algorithms do not have knowledge of the traps or item position. The Greedy agent still achieved a good result because it is able to dodge the traps when it has the updated map since it knows the position of the traps in every round of the game. when the map is configured as a fixed world

4.2 Performance in Dynamic Worlds

In the dynamic world without traps, the Greedy Agent continued to perform well, efficiently collecting items, although the unpredictability of the grid's layout changes slightly impacted its ability to plan ahead. The Random and Snake Agents showed moderate stability, with minor fluctuations in scores based on the number of items they could opportunistically collect before encountering grid changes.

In the dynamic world with 10 traps, all agents faced significant challenges. The Greedy Agent, which previously dominated, experienced substantial performance drops, particularly when operating alone with commitment 20, as its direct movement towards items led it into dangerous situations more frequently. The Random and Snake Agents, although still achieving lower scores overall, suffered comparatively less from the traps, since their erratic or structured movements reduced the likelihood of consistently stepping into hazards.

4.3 Pros and Cons of Each Agent

Each agent exhibited strengths and weaknesses depending on the environment:

- **Greedy Agent:** Highly efficient in collecting items in environments without traps, making it

the best performer when it is aware of the real map, that is, when the commitment is 1 or when the map is a fixed world. However, it struggles significantly in trap-laden environments with higher commitments and dynamic world due to its direct movement strategy, which increases the likelihood of falling into hazards and looking for an item which position had changed in previous rounds.

- **Random Agent:** Its unpredictability gives it some resilience in environments with traps, but it lacks efficiency when item collection is the primary goal. While it avoids falling into patterns, it does not actively optimize its movements.
- **Snake Agent:** Provide stable and predictable performance. However, it lacks the adaptability to react to changes in the environment, which makes it inefficient when items and traps are not properly placed along its movement pattern.

4.4 Effect of Commitment Parameter

The commitment parameter influences how long an agent updates the state of the environment. From the results, it is evident that: - A high commitment (20) makes the Greedy Agent more prone to falling into traps, as it commits too strongly to its path, making it unable to react dynamically to new dangers. - The Random and Snake Agents are not affected by commitment, as their behaviors are inherently less focused on path optimization.

4.5 Best Overall Agent

Based on the results, the Greedy Agent demonstrates the best overall performance in fixed-world scenarios or with commitment equal to 1. However, its main weakness is its susceptibility to traps with higher commitments. A combination of strategies or an improved adaptive behavior would be necessary to achieve consistently high scores.

